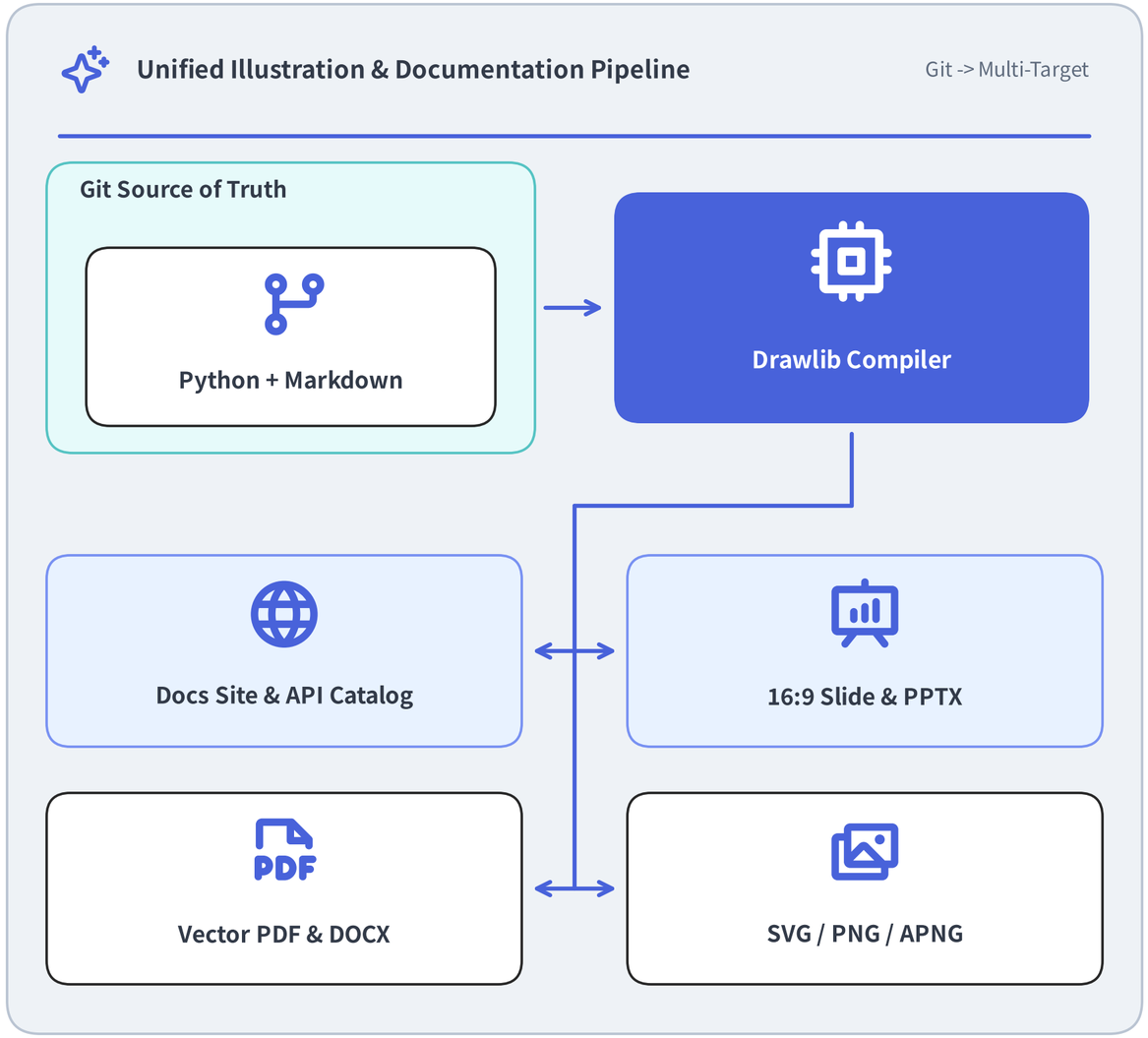
# Drawlib

## Illustration & Documentation as Code

Pure-Python declarative visuals and multi-target documentation compiler built for software engineers and AI coding agents.

**Official Showcase Deck · v0.3.10 Ready**



# Drawlib at a Glance

**4,500+**

**Vector Icons**

Across 34 official & neutral suites

**39+**

**High-Level Visuals**

18 SmartArts, 10 Charts,
6 Diagrams, 5 Graphs

**5**

**Project Archetypes**

site, catalog, doc, slide, images

**100%**

**Pure Python & Git**

Zero GUI lock-in, AI self-verifiable

## Four-Layer Architecture Stack

From AI CLI & Primitives to Multi-Target Publishing

**L4 · Layer 4 · Multi-Target Compiler**

HTML · Catalog · Slide · PDF · DOCX · PPTX

**L3 · Layer 3 · Declarative Visual Engines**

SmartArts · Charts · Diagrams · Graphs · Anim

**L2 · Layer 2 · Primitives, 34 Icon Suites & Semantic Styles**

43 Shapes · 4,500+ Vector Icons · Theme Presets

**L1 · Layer 1 · AI Agent CLI**

rules · show -g Grid Inspection · SQLite Cache

# Presentation Roadmap

## Six Core Chapters

- **Motivation (Ch. 1)**: Solving visual documentation drift with Python and Git
- **Visual Engines (Ch. 2–4)**: Primitives, 34 icon suites, SmartArts, charts, and diagrams
- **Publishing & AI (Ch. 5–6)**: Five project archetypes, Office export, and AI grid review



CHAPTER

01

Chapter 1 of 6

# Why Drawlib?

Solving Visual Documentation Drift in the AI Era


1.1 Why GUI Whiteboards & Text DSLs Break Down

1.2 Illustration & Documentation as Code

1.3 Landscape Comparison Matrix

1.4 Single Source of Truth Across Formats

# Why Visual Documentation Breaks Down

Engineering teams are forced to choose between **untrackable GUI binaries**, **rigid text DSLs**, and **isolated plotting libraries**—causing visual architecture docs to drift behind code on every release.

### GUI Tools (Figma / Draw.io)

- **Opaque Binary Files**
  No Git PR diff review
- **Manual PNG Exports**
  Screenshots rot on release
- **GUI Lock-In**
  Uneditable by AI agents

### { } Text DSLs (Mermaid / UML)

- **Rigid Auto-Layout**
  Zero exact (x, y) control
- **Narrow Vocabulary**
  Weak KPIs, tables & maps
- **Snippet-Only Scope**
  No PDF or PPTX compiler

### Plotting Libs (Matplotlib)

- **Data Plots Only**
  No cloud or UML engines
- **Verbose Boilerplate**
  Ad-hoc fonts and colors
- **Detached Scripts**
  Manual copy into slides

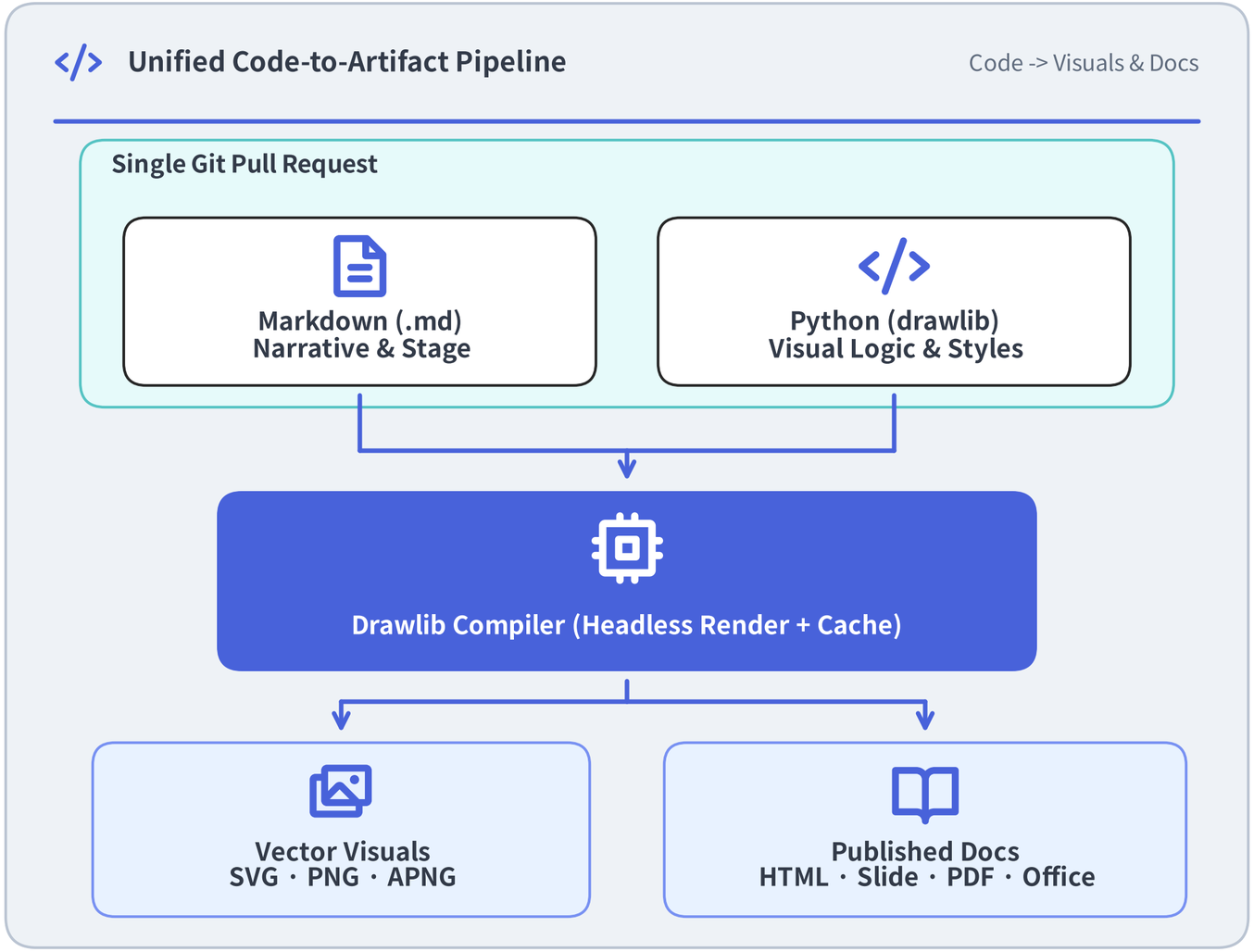
**The Unmet Gap: Spatial Coordinates + Git PR Review + Unified Docs & Charts + AI Grid Verification**

# Illustration & Documentation as Code

## Pure Python + Markdown

- **Turing-Complete Layout**: Loops, variables, and shared `utils.py` helpers
- **Inline Code Blocks**: Embed `drawlib` visuals directly inside Markdown
- **Deterministic PR Diffs**: Review prose and diagram updates together

```
setup(width=120, height=32)
p = ChevronProcess(style=Styles.Neutral)
for s in ("Code", "Build", "Ship"): p.add(s)
p.draw((6, 8), width=108, height=16)
save()
```

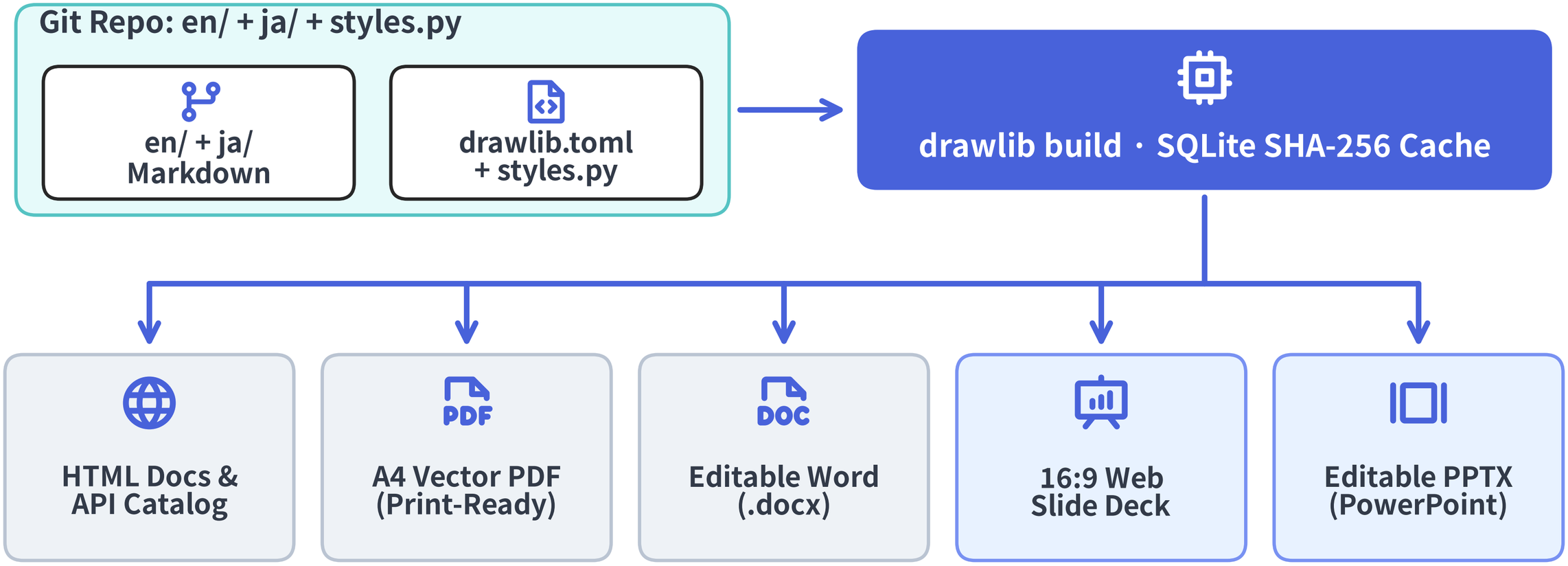


# Tooling Landscape Comparison

| Capability | GUI Whiteboards | Text DSLs | Plotting Libs | Drawlib |
|---|---|---|---|---|
| **Git Diff & PR Review** | × Binary Blob | ✓ Text Syntax | ✓ Python Code | **✓ Clean Python** |
| **Spatial Coordinate Control** | ✓ Manual Drag | × Rigid Auto | △ Axes Only | **✓ Exact + Auto** |
| **Cloud & UML Diagrams** | ✓ Manual Icons | △ Basic Boxes | × Unsupported | **✓ 6 Engines + Icons** |
| **Charts & Infographics** | △ Manual Shapes | × Very Limited | ✓ Plots Only | **✓ Charts & SmartArts** |
| **Site / Catalog / Slide / Office** | × Image Export | × Snippet Only | × Image Only | **✓ 5 Archetypes** |
| **AI Visual Self-Healing (-g)** | × GUI Locked | × Blind Text | △ Ad-Hoc Scripts | **✓ Built-In -g CLI** |

# One Git Repository, Every Output Format

A single `drawlib.toml` manifest with `[i18n]` structural parity compiles synchronized **English and Japanese** deliverables across Web, Print, and Office from one Git commit.



CHAPTER

02

Chapter 2 of 6

# Primitives, Styles & Icons

43 Shapes, Semantic Themes & 4,500+ Vector Icons


2.1 Cartesian Canvas & -g Visual Grid

2.2 43 Shapes & Smart Connectors

2.3 Multilingual Typography & Fonts

2.4 34 Icon Suites & 8-Color Themes

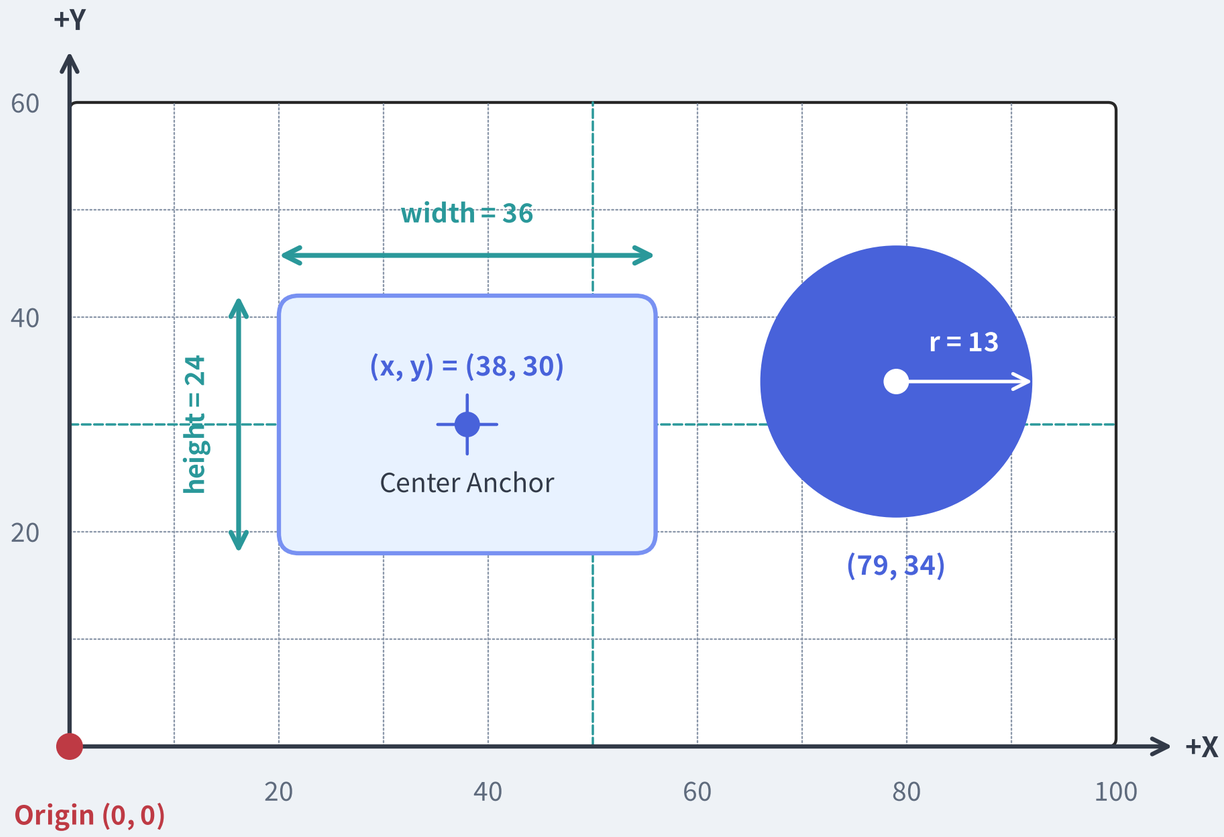
# Cartesian Canvas & Visual Grid (-g)

## Predictable Math Geometry

- **Bottom-Left Origin**: `(0, 0)` starts bottom-left (`+X` right, `+Y` up).
- **Center Anchor**: `(x, y)` centers every shape, text, and icon.
- **Visual Grid ( `-g` )**: Overlay coordinate ticks for exact verification.

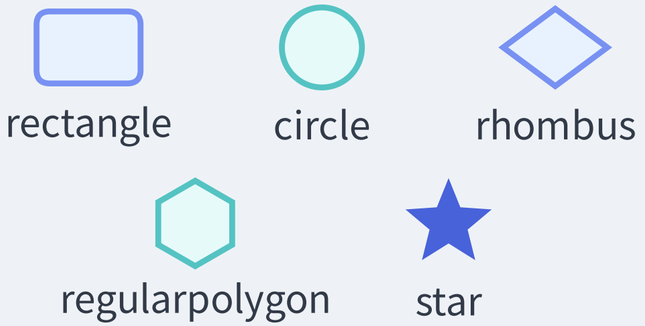
```
setup(width=100, height=60)
st = Styles.PrimaryFlat
rectangle((50, 30), 36, 22, style=st)
save()
```

**Cartesian Stage: setup(width=100, height=60) + Grid (-g)**



# 43 Geometric Shapes & Smart Connectors

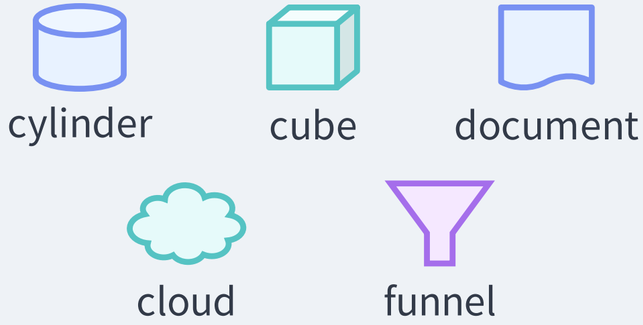
## 1. Basic & Polygons



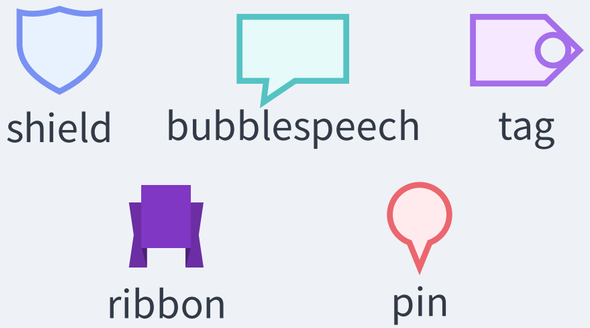

## 2. Block Arrows & Flow



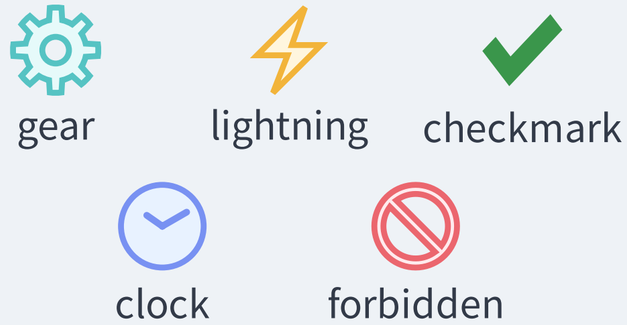

## 3. 3D & Containers



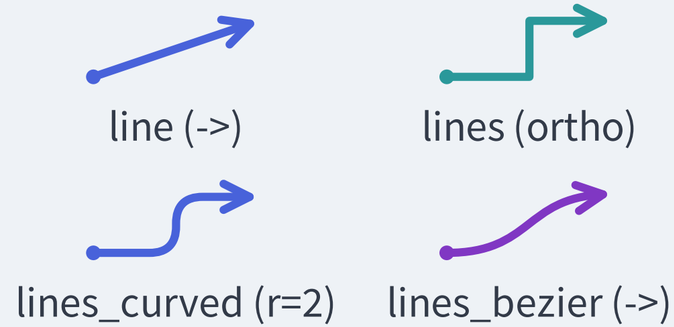

## 4. Badges & Callouts




## 5. Domain Symbols



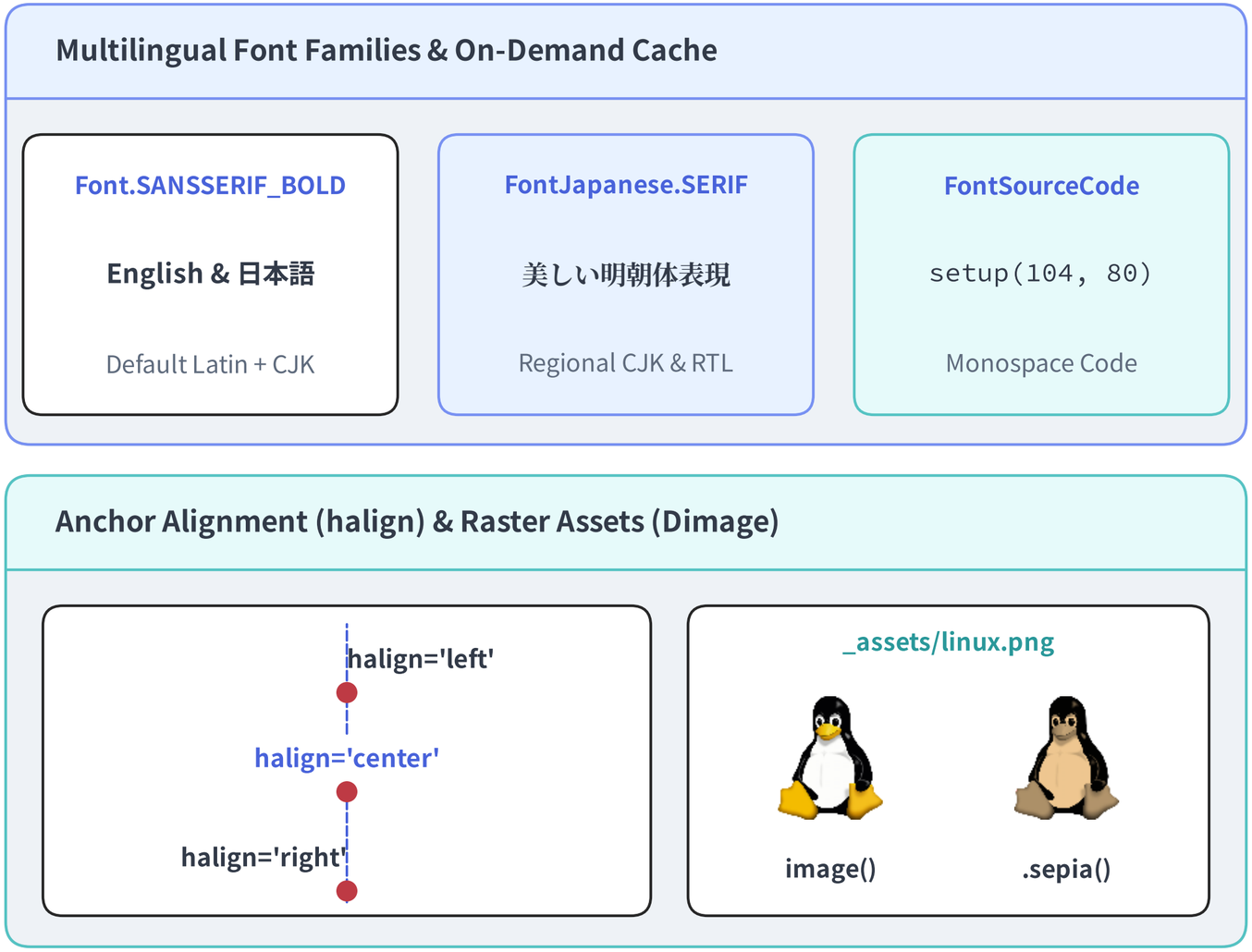

## 6. Lines & Curves



# Multilingual Typography & Raster Assets

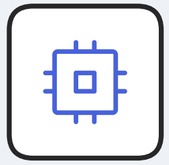
## Universal Scripts & Dimage

- **CJK + Latin Default**: `Font` renders English and Japanese together.
- **Regional Packs**: `FontJapanese`, `FontArabic`, and `FontSourceCode`.
- **Exact Text Anchors**: Control alignment via `halign` and `valign`.
- **Raster** `Dimage`: Crop, resize, sepia, mosaic, plus font caching.



# 34 Icon Suites & 4,500+ Vector Icons

## 1. Phosphor Vector Font Icons (5 Weights)

| thin | light | regular | bold | fill |
|---|---|---|---|---|
|  |  |  |  |  |
| rocket_launch | cpu | globe | sparkle | shield_check |

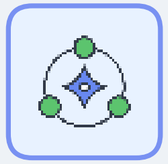
## 2. Official Cloud Suites (gcp · aws · azure)

| gcp | gcp | aws | aws | azure |
|---|---|---|---|---|
|  |  |  |  |  |
| Cloud Run | BigQuery | Lambda | Amazon S3 | Cosmos DB |

## 3. 15 Official Domain Brand Suites

| ai | cnative | devops | data | security |
|---|---|---|---|---|
|  |  |  |  |  |
| Gemini | Kubernetes | GitHub | PostgreSQL | Cloudflare |

## 4. 15 Original 2D Suites (original_*)

| cloud | ai | server | network | security |
|---|---|---|---|---|
|  |  |  |  |  |
| multi_cloud | ai_agent | gpu_server | firewall | zero_trust |

# 8-Color Semantic Styles & Preset Themes

- **Semantic Matrix**: `Styles.<Color><Variant>` combines 8 roles ( `Primary` .. `Dark` ) with `Flat` , `Bold` , `Dashed` , and `Neutral` .
- **Immutable `.patch()` & Themes**: Customize safely via `style.patch(...)` or switch global themes in `drawlib.toml` .

## 8 Semantic Roles × Variants (Styles.<Color><Variant>)

| Role | Default | Flat | Neutral / Dashed |
|---|---|---|---|
| Primary | .Primary | .PrimaryFlat | .PrimaryNeutral |
| Secondary | .Secondary | .SecondaryFlat | .SecondaryNeutral |
| Accent | .Accent | .AccentFlat | .AccentNeutral |
| Success | .Success | .SuccessFlat | .SuccessNeutral |
| Warning | .Warning | .WarningFlat | .WarningNeutral |
| Danger | .Danger | .DangerFlat | .DangerNeutral |
| Neutral | .Neutral | .NeutralFlat | .NeutralDashed |
| Dark | .Dark | .DarkFlat | .DarkDashed |

## Preset Themes (drawlib.toml)

**style = "default"**
Indigo Editorial Palette



**style = "google"**
Google Workspace Blue



**style = "monochrome"**
Print & Paper Grayscale



CHAPTER

03

Chapter 3 of 6

# SmartArts & Pure-Python Charts

18 Structured Infographics & 10 Zero-Dependency Plots


3.1 Process Flows, Cycles & Roadmaps

3.2 KPIs, CardLists, Pyramids & MindMaps

3.3 Tables, SourceCode, Calendars & GeoMap

3.4 10 Declarative Chart Types & Gantt

# Process Pipelines, Lifecycles & Roadmaps

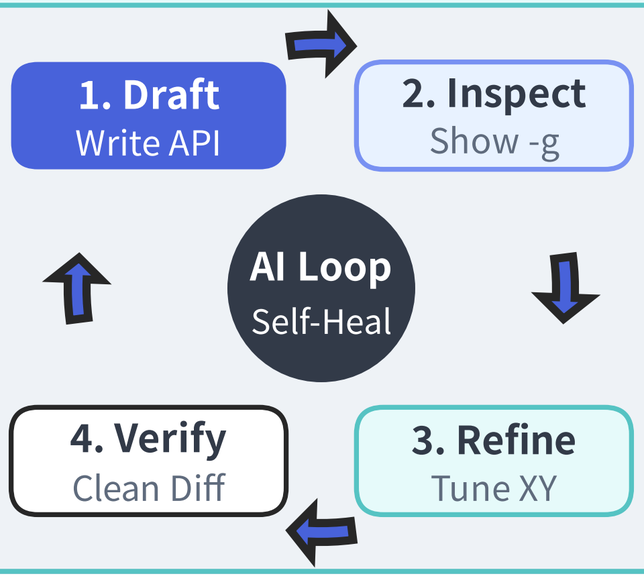
## ChevronProcess — Sequential Delivery Pipeline

4 Stages

1. **Author** — Markdown + Python
2. **Render -g** — Visual Grid Check
3. **Cache** — SQLite Incremental
4. **Publish** — HTML, PDF & PPTX

## Cycle — Feedback Loop

4-Node Orbit




## Roadmap — Milestone Timeline

Bezier Spine

- **P1** — Core Vector Canvas: Primitives, icons & theme tokens
- **P2** — SmartArts & Charts: 18 infographics & 10 declarative plots
- **P3** — Diagrams & Graphs: 6 diagram engines & layout solvers
- **P4** — Multi-Format Export: HTML, PDF, PPTX & AI rules CLI

# KPIs, Feature Cards, Pyramids & MindMaps

## KpiCards — Metric Grid

2x2 Layout

| | |
|---|---|
| 18 | **SmartArt Types** Tables to GeoMaps |
| 34 | **Icon Suites** Cloud & Phosphor |
| 0 | **GUI Lock-In** 100% Git Python |
| 4 | **Theme Palettes** Instant re-skin |

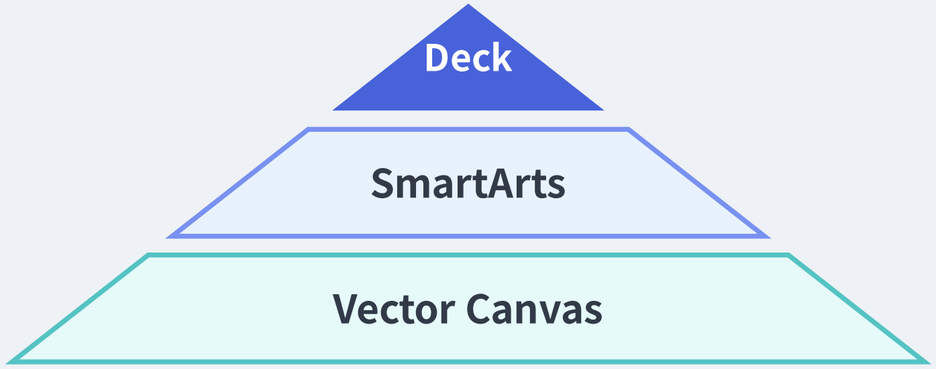
## CardList — Feature Stack

Icon + Badge

- L1 **Declarative Item Lifecycle** Register via .add() and chain .draw()
- L2 **Semantic Design Tokens** 50%+ neutral base with focal highlights
- L3 **Animation-Ready Visibility** Toggle item.show without shifting layout

## Pyramid — Layered Hierarchy

3 Tiers




## MindMapNode — Radial Tree

2-Pass Layout



# Tables, SourceCode, Calendars & GeoMap

## Table — Benchmark Matrix

Zebra Rows

| Output Target | Format | Cold Build |
|---|---|---|
| 16:9 Slide Deck | HTML / PDF | **0.42 s** |
| Documentation Site | Multi-Page | **0.65 s** |
| Standalone Diagram | SVG / PNG | **0.18 s** |

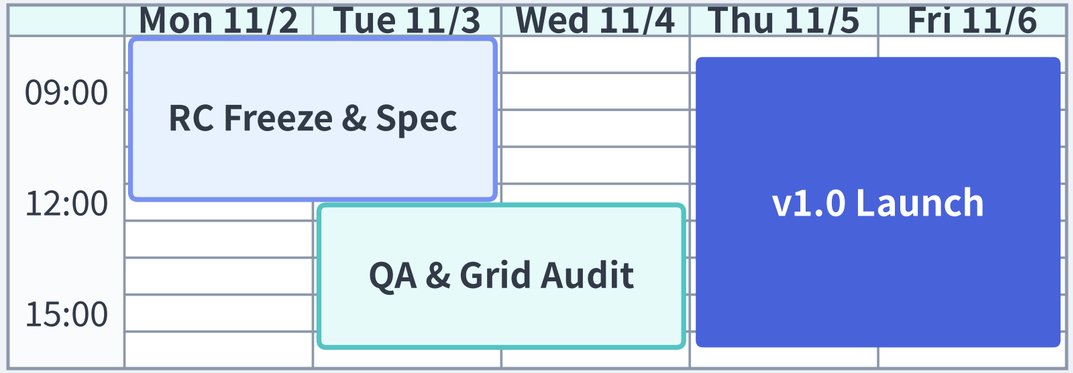
## SourceCode — Syntax Box

card_demo.py

```python
from drawlib.canvas import save, setup
from drawlib.shapes import rectangle
from drawlib.styles import Styles
setup(width=120, height=60)
rectangle((60, 30), 80, 36, Styles.Primary)
save("card.svg")
```

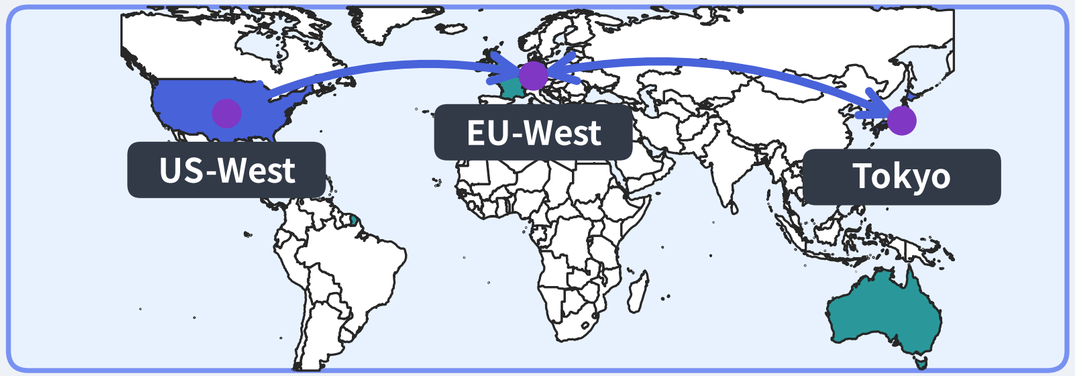
## WeekCalendar — Sprint Plan

ISO 8601




## GeoMap — Global Regions

US, EU & JP



# Declarative Bar, Line & Area Charts

**10**

**Built-In Chart Types**

Bar, Box, Line, Area, Pie,
Radar, Scatter, Gantt, Heatmap

**100%**

**Theme-Synchronized**

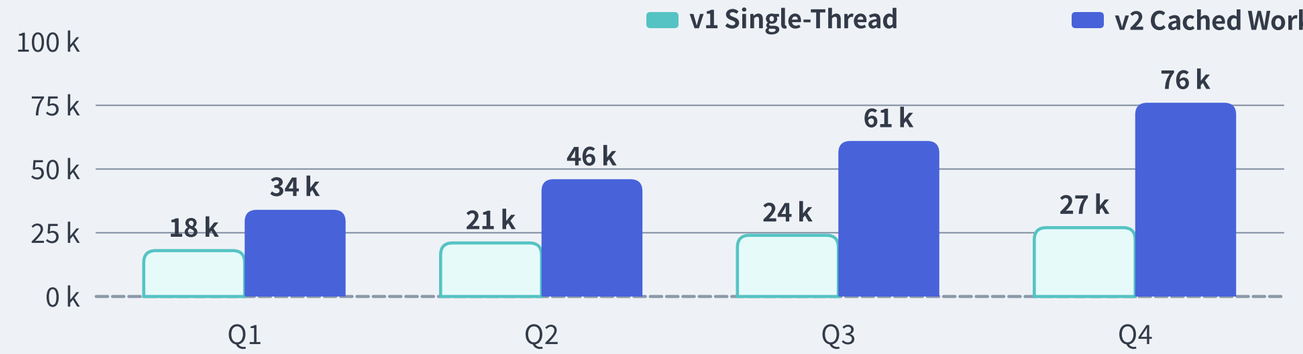
Matches Default, Google &
Monochrome palettes

**Auto**

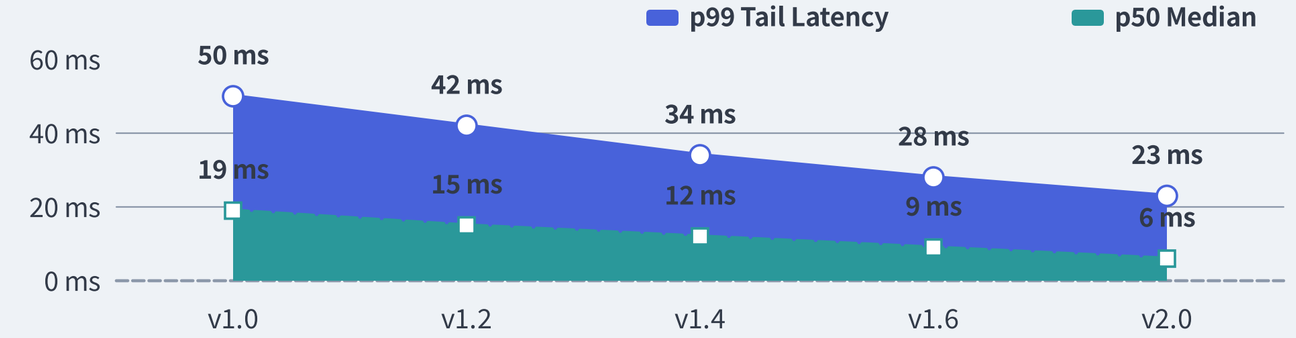
**Nice-Number Axes**

Clean ticks, legends &
progressive draw_ratio

**BarChart — Quarterly Render Throughput (k/hr)** — Grouped Mode



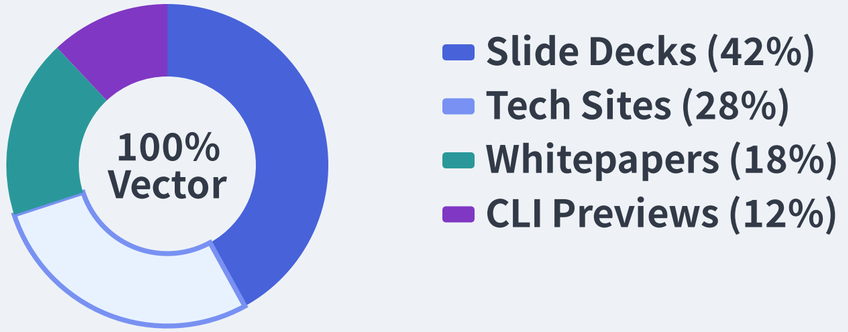

**AreaChart — Build Pipeline Latency (p50 vs p99)** — Smooth Spline



# Pie, Radar, Scatter & Heatmap Charts

## PieChart — Donut Share

hole_ratio=0.55



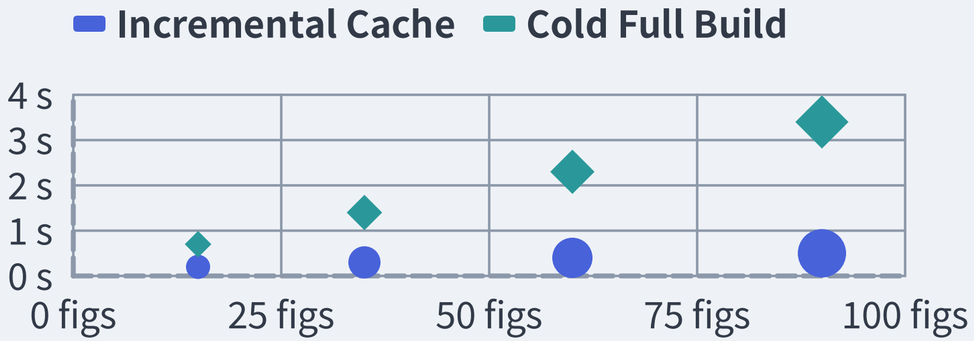

## RadarChart — 5-Axis Profile

Polygon Web




## ScatterChart — Build Scaling

Bubble Series



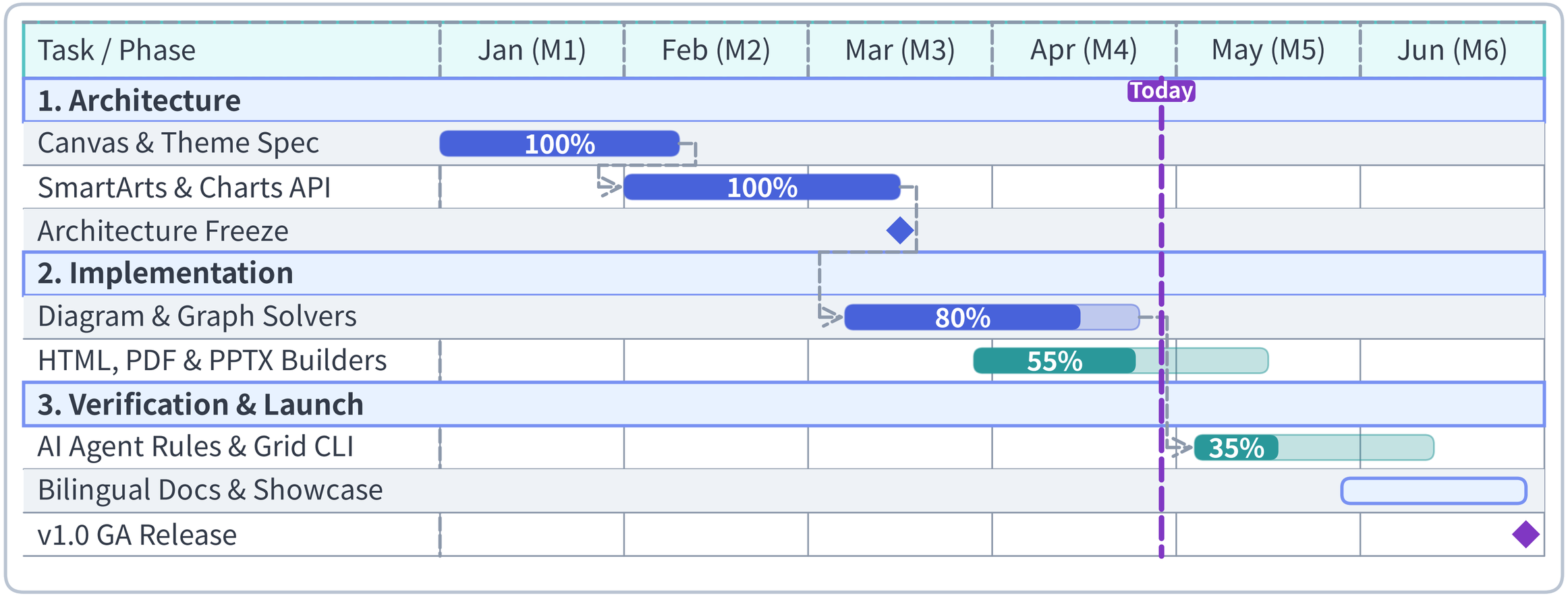

## HeatmapChart — Latency

Auto-Contrast

| | 00:00 | 06:00 | 12:00 | 18:00 | 22:00 |
|---|---|---|---|---|---|
| US-West | 14ms | 18ms | 42ms | 35ms | 19ms |
| EU-West | 16ms | 29ms | 54ms | 38ms | 21ms |
| Tokyo | 22ms | 46ms | 68ms | 49ms | 25ms |
| Singapore | 18ms | 34ms | 51ms | 41ms | 22ms |

# Engineering Roadmaps with GanttChart

- **Declarative Schedule Modeling**: Group workstreams with `Section`, track completion via `Task(progress=...)`, and pin `Milestone` diamonds.
- **Dependencies & Live Status Markers**: Route orthogonal `Dependency` arrows and overlay vertical `Marker` lines (`Today`, `Release`) in code.



CHAPTER

04

Chapter 4 of 6

# Diagrams, Graphs & Animations

6 Engineering Diagram Engines, 5 Solvers & APNG

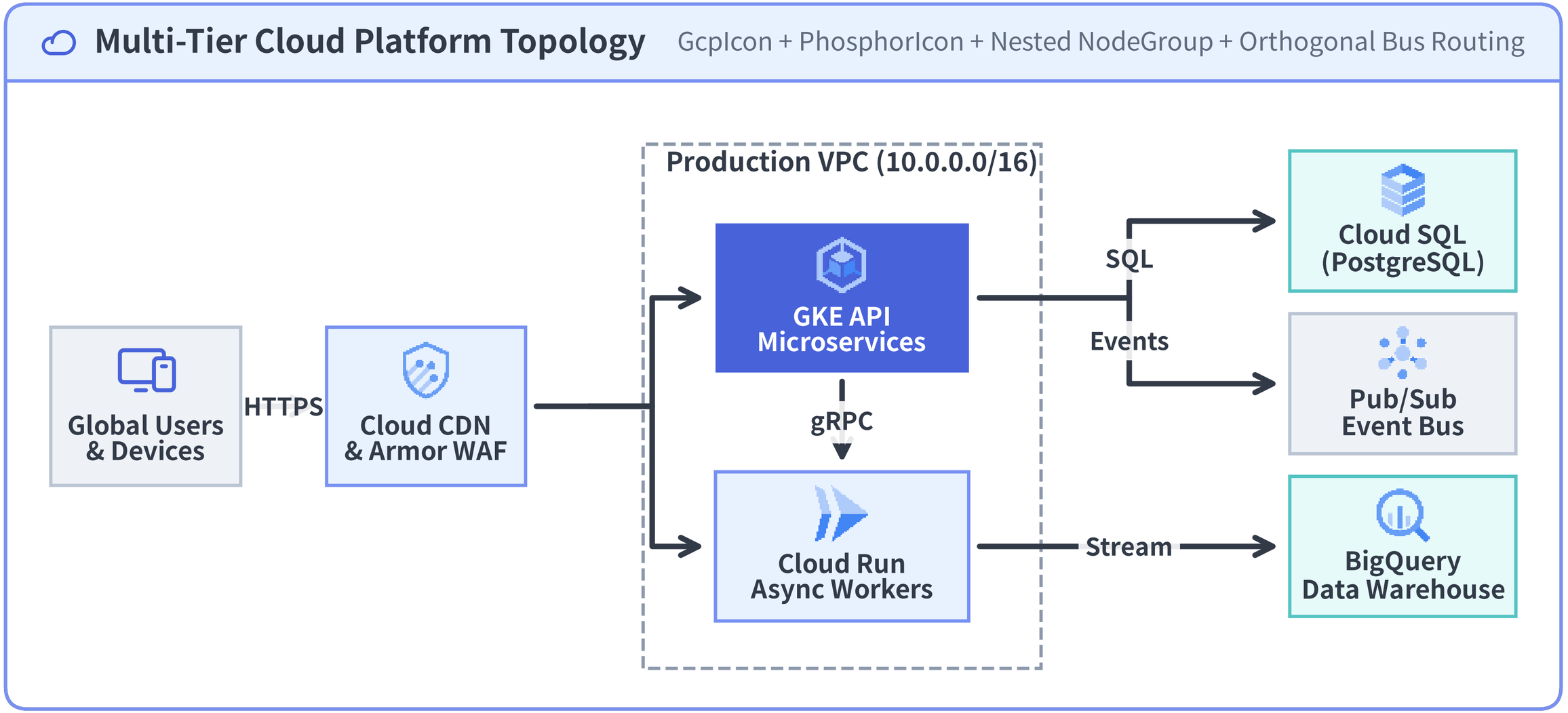

4.1 Cloud Topology (ArchitectureDiagram)

4.2 Flowcharts & API Sequence Diagrams

4.3 UML Class, ER & State Machines

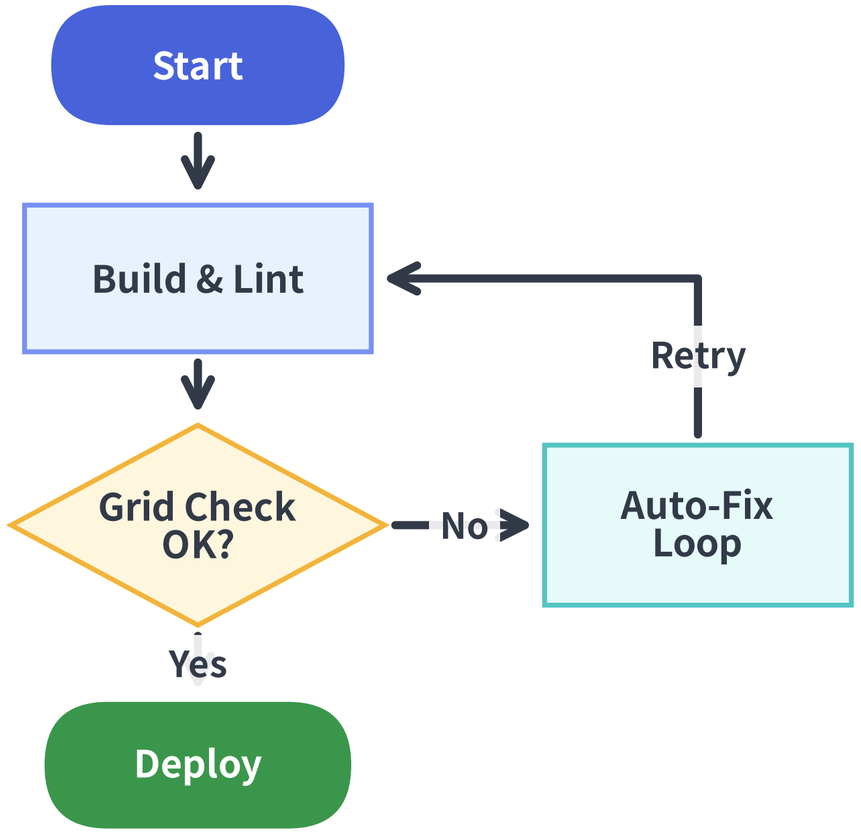
4.4 Auto-Layout Graphs & Animations

# Cloud Topology with ArchitectureDiagram

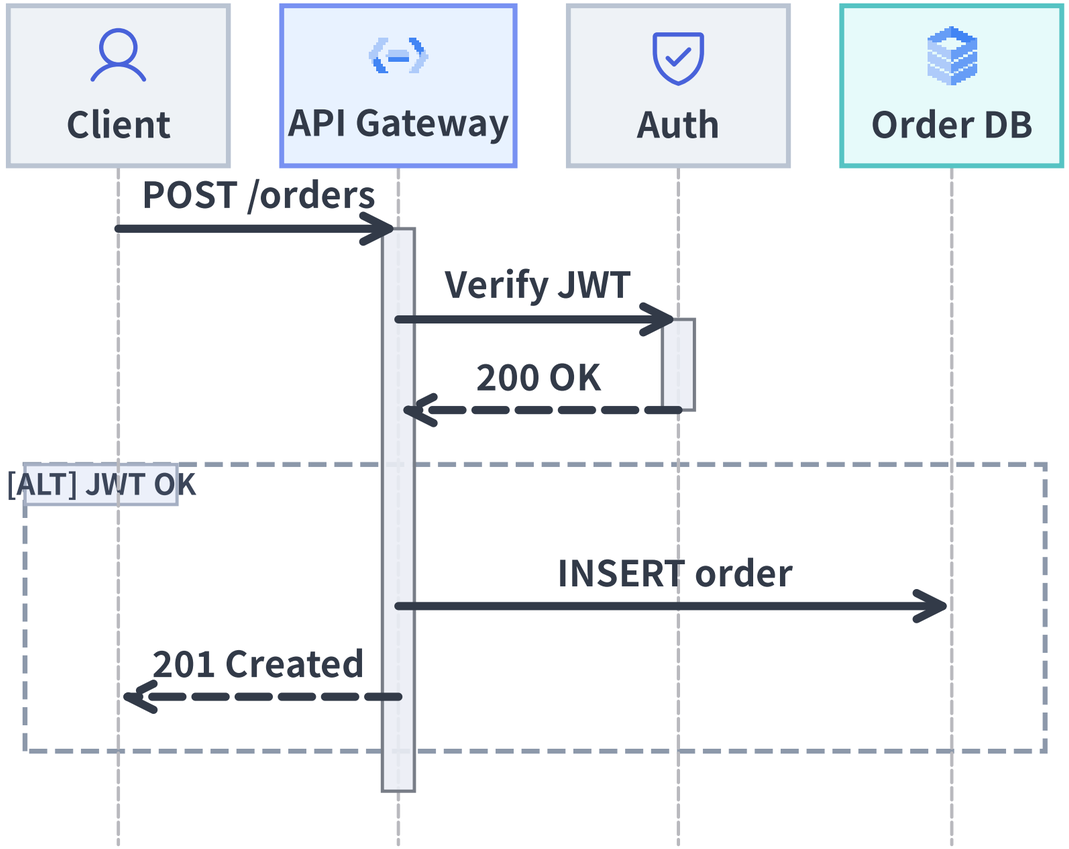


# Workflows & Protocols: Flow and Sequence

## FlowDiagram — CI/CD Validation

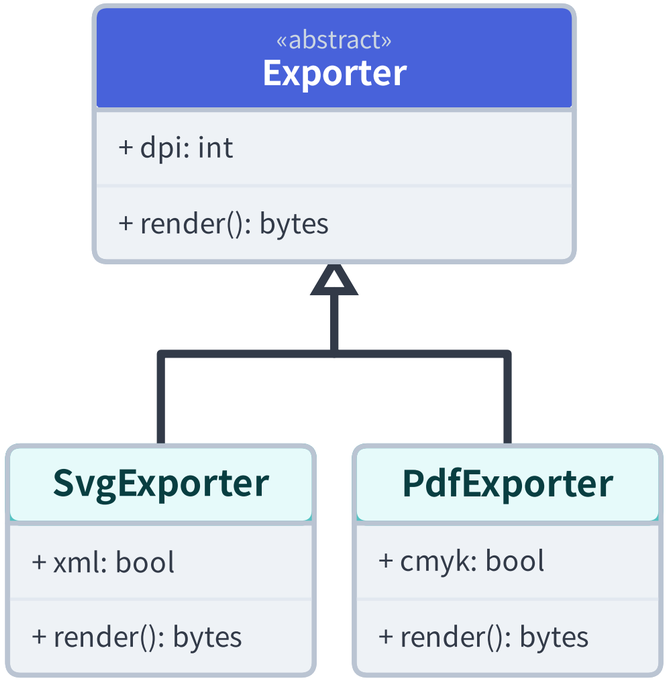



## SequenceDiagram — API Protocol



# Domain Modeling: Class, ER & State Diagrams

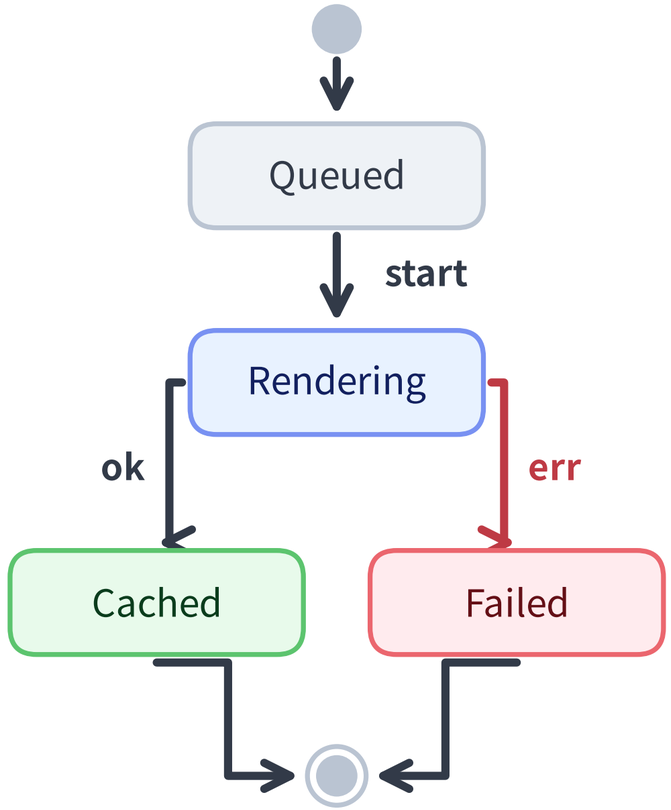
## ClassDiagram (UML 2.0)




## ERDiagram (Schema)




## StateDiagram (FSM)

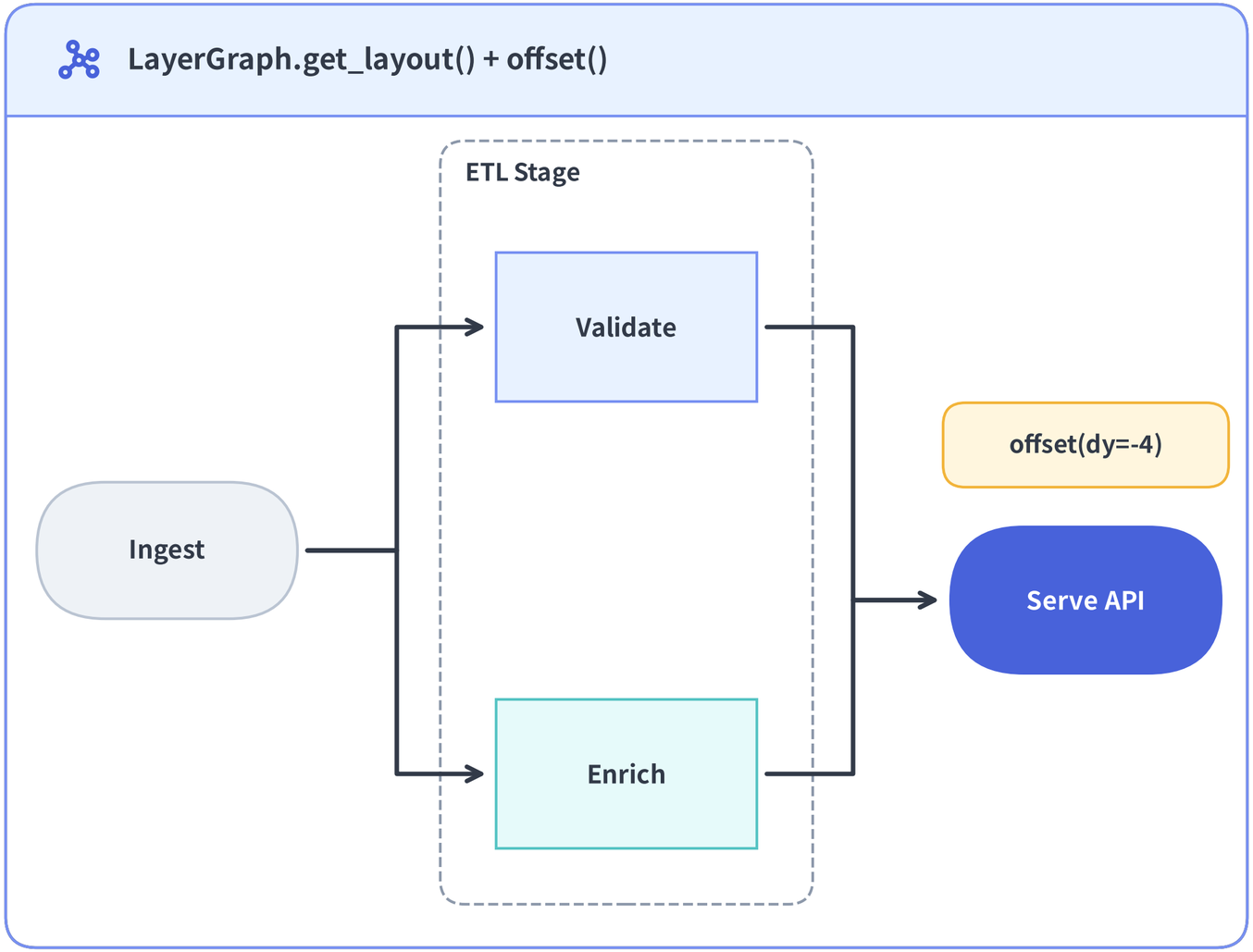


# Declarative Graph Layout (drawlib.graph)

## Hybrid Algorithmic + Manual Control

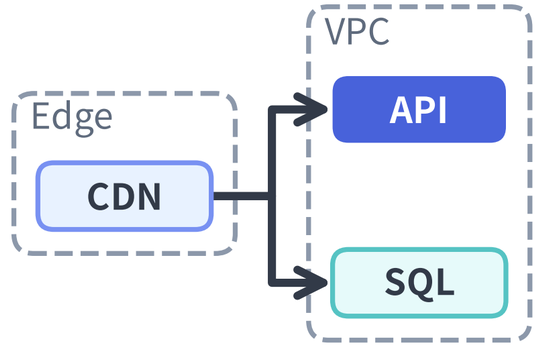
- **Auto-Solve Topology**: Crossing-free `(x, y)` via `get_layout()`
- **Manual Fine-Tuning**: Nudge nodes with `offset(dx, dy)` anytime
- **No Black-Box Lock-In**: Inspect & freeze code via `print(layout)`

```
g = LayerGraph(direction="LR")
g.edge("ingest", "validate")
g.edge("validate", "serve")
layout = g.get_layout(margin=8.0)
layout.offset("serve", dy=-4.0)
```

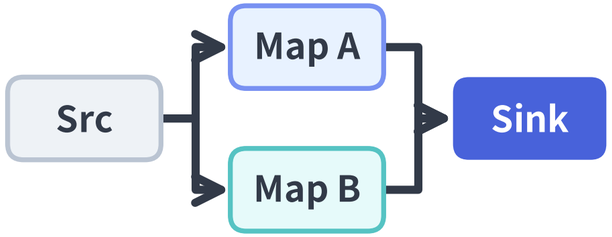


# Five Specialized Auto-Layout Graph Solvers

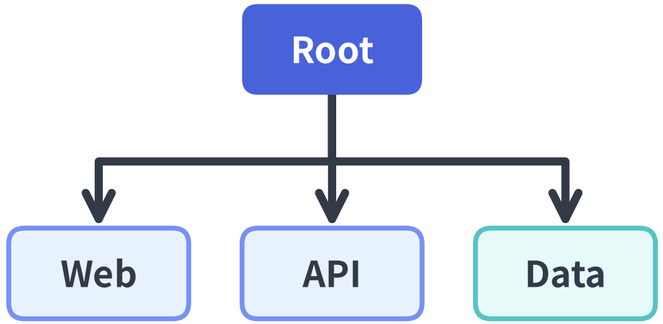
## 1. ArchitectureGraph



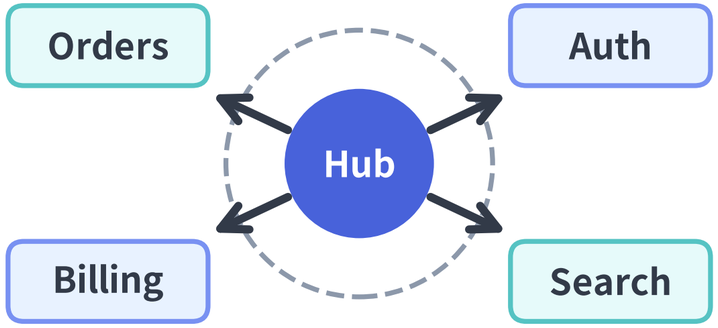

## 2. LayerGraph (DAG Tiers)



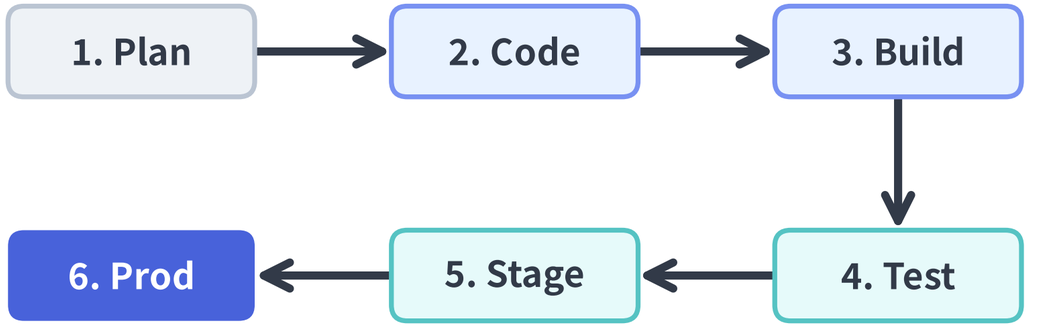

## 3. TreeGraph (Tidy Tree)




## 4. RadialGraph (Hub & Concentric Spokes)




## 5. GridGraph (2D Matrix & Snake Flow)

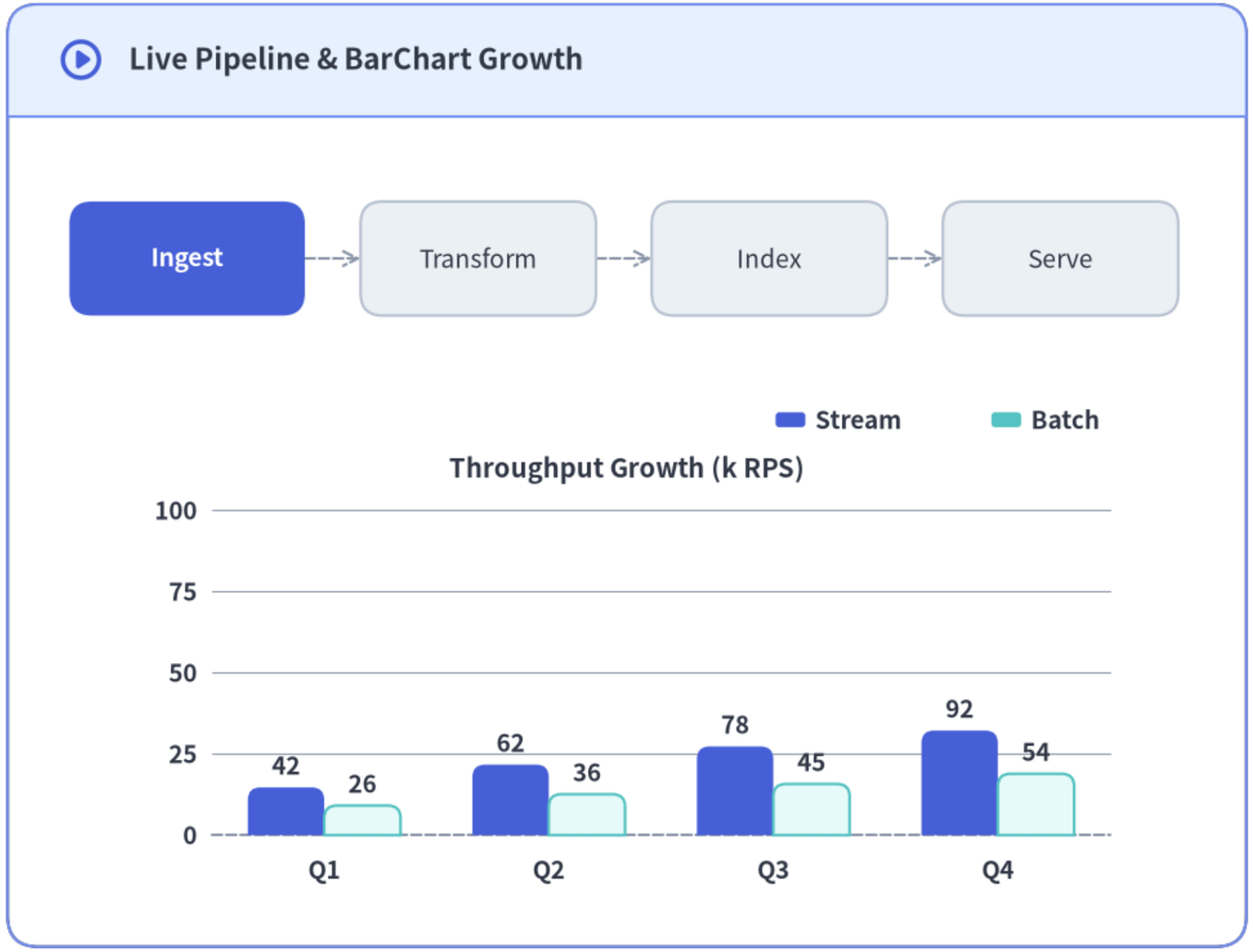


# Interactive Frame Animations (APNG & WebP)

## Motion as Code Across All Modules

- **Multi-Frame Engine**: Declarative `Animation(fps, loop)` API
- **Progressive Reveal**: `draw_ratio` on Lines, Charts & Diagrams
- **Interactive Stage**: Live `<canvas>` ( `anim_trigger` , `anim_pause` , `A` )
- **Static Export Ready**: Auto-extracts clean frames for PDF & PPTX

```
anim = Animation(fps=2.0, loop=0)
for r in [0.4, 0.7, 1.0]:
    series.draw_ratio = r
    with anim.frame():
        chart.draw(xy=(10, 6))
```



CHAPTER

05

Chapter 5 of 6

# 5 Archetypes & Publishing

Site, Catalog, Doc, Slide & Images from One Compiler


5.1 Five Project Archetypes Overview

5.2 Web Docs (site) & API Reference (catalog)

5.3 Whitepapers: HTML, Vector PDF & DOCX

5.4 16:9 Slide Stage, Presenter View & PPTX

# Five Built-In Project Archetypes

`uv run drawlib init project <type> <dir>` — One Compiler • 5 Purpose-Built Scaffolds • Web, PDF & Office

## site

`docs_src/<lang>/`

Guided narrative docs
navbar.md + search
Dark mode & i18n

e.g. www.drawlib.com

HTML | Search | SVG

## catalog

NEW in v0.3.10

`catalog_src/<lang>/`

Two-column API ref
Auto # H1 / ## H2
Instant / filter

e.g. drawlib.com/api

HTML | / Filter | SVG

## doc

`doc_src/<lang>/`

Linear whitepapers
--generate-index
Auto Cover & ToC

RFCs & Design Specs

HTML | PDF | DOCX

## slide

`slide_src/<lang>/`

16:9 visual stage
?presenter=1 sync
::: note scripts

Keynotes & Tech Talks

HTML | PDF | PPTX

## images

`images_src/*.py`

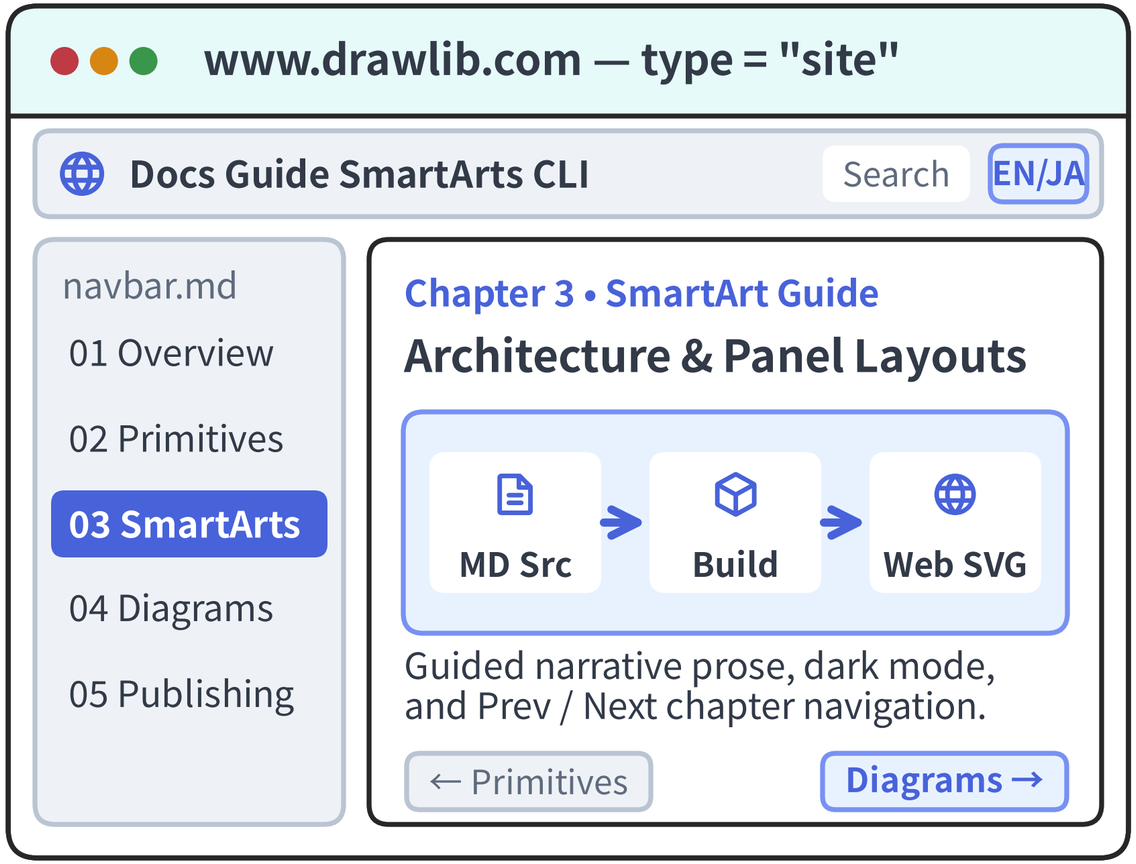
Standalone Python
Batch illustration
CI asset generation

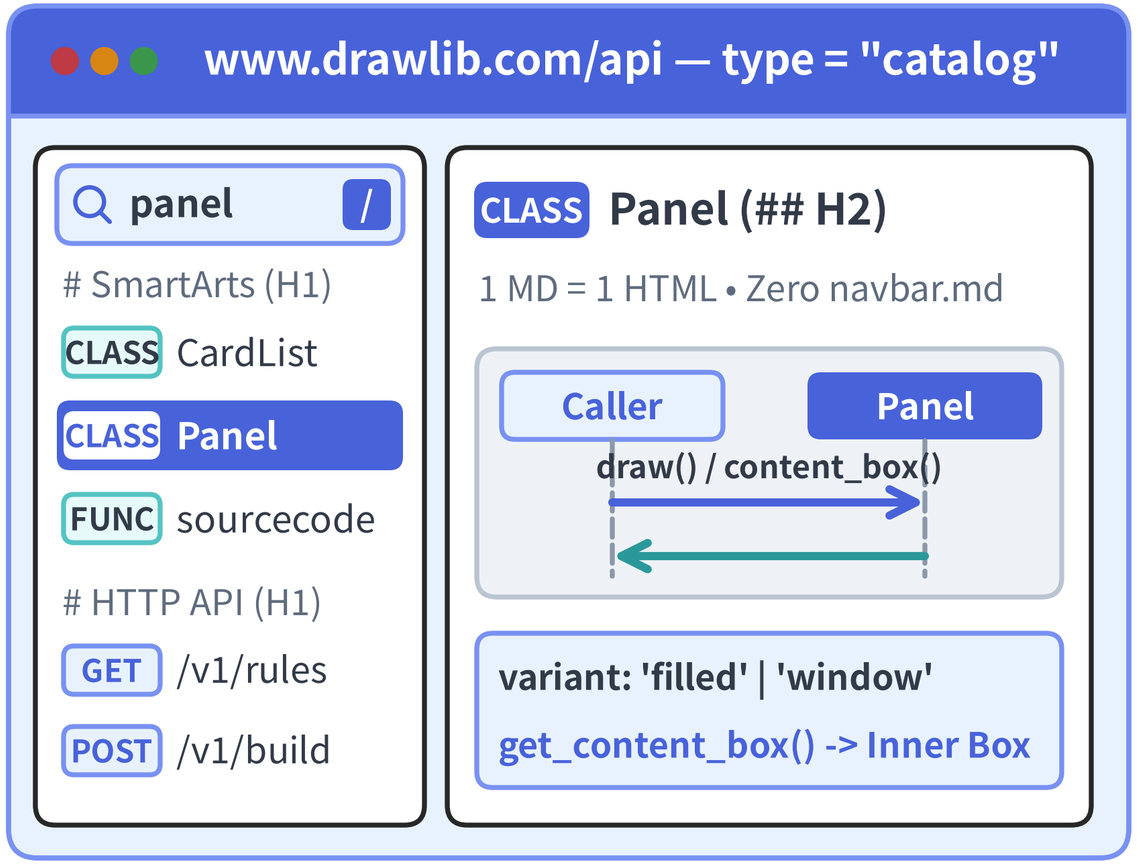
README & Paper Art

SVG | PNG | WebP

# Web Publishing: Docs Site & API Catalog

- `type = "site"` **(Narrative Docs)**: Guided chapter navigation via `navbar.md`, full-text search, dark mode & `i18n` (`www.drawlib.com`).
- `type = "catalog"` **(API Reference)**: Zero-`navbar.md` `1 MD = 1 HTML` dictionary extracting `# H1` / badged `## H2` items with `/` instant filter (`www.drawlib.com/api`).



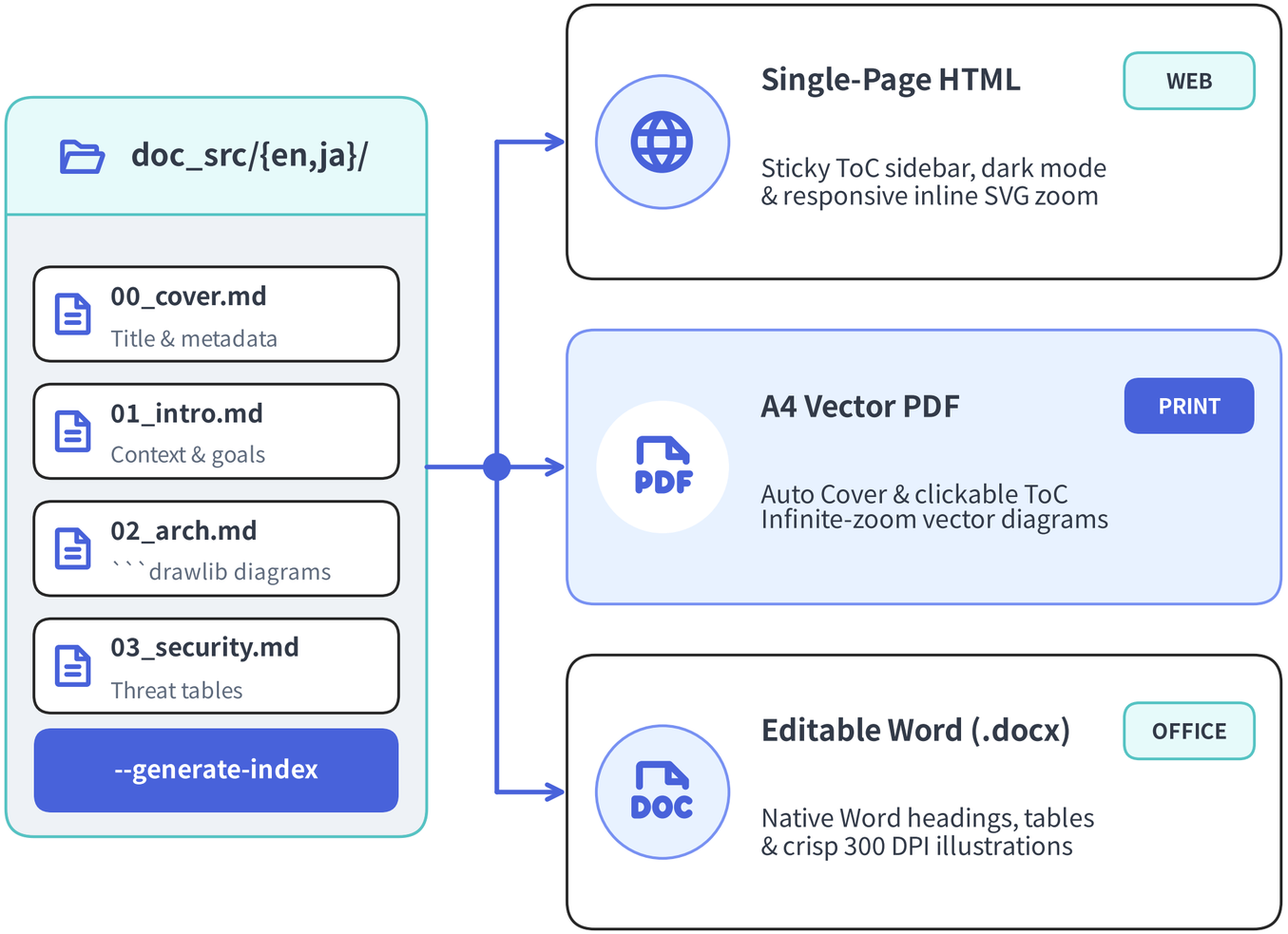



# Whitepapers: HTML, Vector PDF & MS Word

## Linear Specs ( `type = "doc"` )

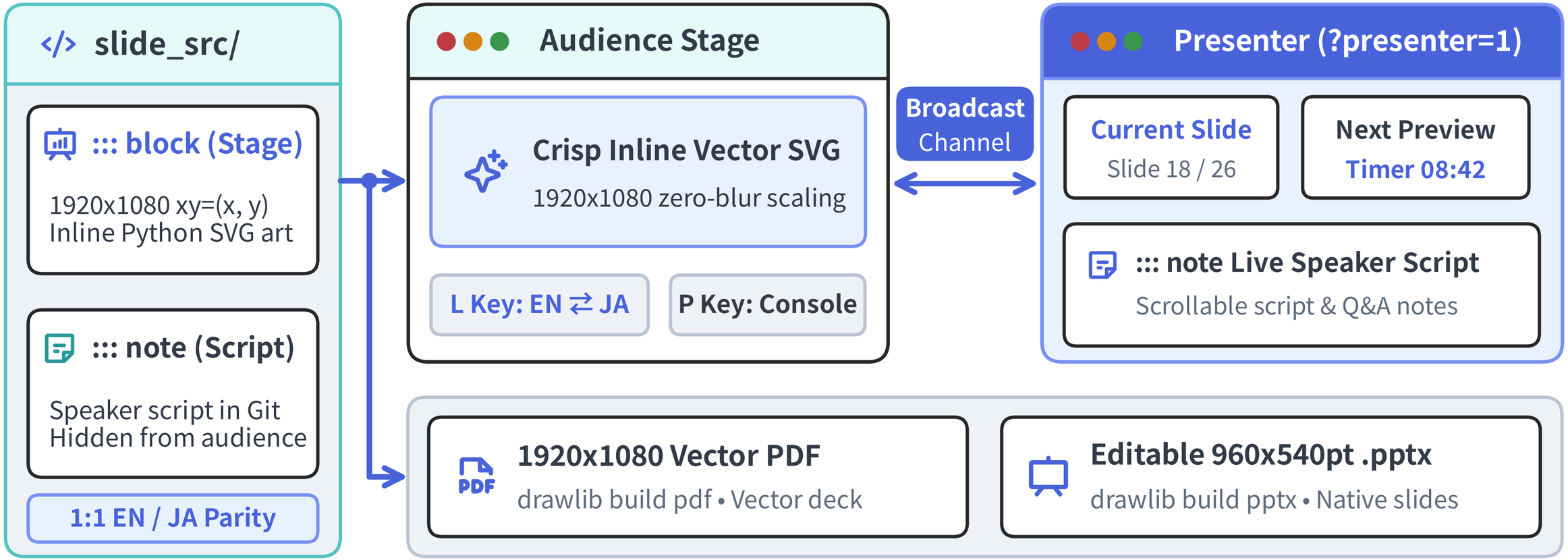
- **Numbered Chapters**: Merge `00_cover.md`, `01_intro.md` into one spec
- **Auto Cover & ToC**: `--generate-index` builds cover & clickable index
- **Triple Export**: Compile HTML, A4 Vector PDF, and editable `.docx`

```
# Build all 3 deliverables from doc_src/
uv run drawlib build html
uv run drawlib build pdf --generate-index
uv run drawlib build docx --generate-index
```



# 16:9 Slide Stage, Presenter View & PPTX

- **Dual-Layer Slide Markdown**: `::: block` places crisp visuals on a `1920×1080` stage while `::: note` stores speaker scripts in Git.
- **Live Sync & Office Export**: Sync Audience Stage with `?presenter=1` Presenter View via `BroadcastChannel`, plus Vector PDF & `.pptx`.



CHAPTER

06

Chapter 6 of 6

# AI-Native Agent Workflow

Rules CLI, Visual Self-Correction & SQLite Caching

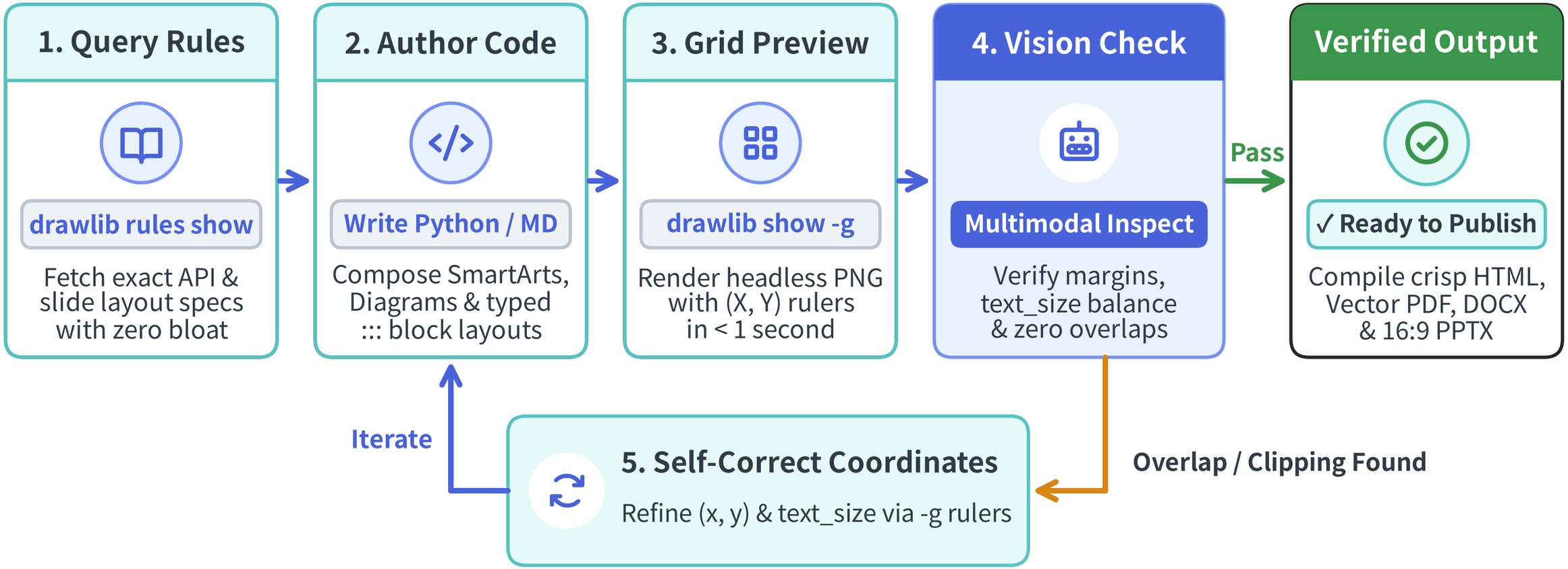

6.1 On-Demand Context via drawlib rules

6.2 Autonomous -g Grid Verification Loop

6.3 Two-Tier Asset & SQLite Build Cache

# Autonomous Multimodal Verification Loop

- **On-Demand Context**: AI agents (Gemini, Claude Code, Cursor) query exact API specs via `uv run drawlib rules show <topic>`.
- **Visual Self-Correction**: Agents inspect `-g` coordinate grid previews multimodally and fix overlaps before publishing.



# Two-Tier Caching & Unified CLI

**< 1s**

**Incremental Builds**

SQLite SHA-256 .drawlib/cache.db

**50+**

**Agent Rule Topics**

On-demand via drawlib rules show

**Zero**

**Broken Links**

Built-in drawlib serve --check

## Two-Tier Cache Architecture

### Tier 1: Asset Cache

**drawlib cache list / download**

On-demand fonts, 34 icon packs
& headless Chromium binary

### Tier 2: Build Cache

**.drawlib/cache.db (SQLite)**

SHA-256 content + style hash
Shared by HTML/PDF/Office

## Unified CLI — uv run drawlib <command>



CLOSING 06 • Wrap-Up

# Summary & Next Steps

Start Building Illustrated Documentation as Code

# Key Takeaways & Quickstart

01

## 1. Pure Python & Git Native

Typed Python API, styles.py tokens & clean Git PR diffs — zero GUI lock-in.

02

## 2. Rich High-Level Components

18 SmartArts, 10 Charts, 6 Diagrams, 5 Graph solvers & 34 Icon Packs built in.

03

## 3. 5 Project Archetypes & Office Export

Compile Site, Catalog, Doc, Slide & Images to HTML, Vector PDF, DOCX & PPTX.

04

## 4. Built for Autonomous AI Agents

50+ drawlib rules topics, show -g grid QA & < 1s SQLite .drawlib/cache.db builds.

**QUICKSTART IN 30 SECONDS**

Requires Python 3.11+ & uv

```
uv add drawlib → uv run drawlib init project slide my-deck → uv run drawlib serve
```

# Thank You

## Start Authoring Visuals & Docs as Code

- **Docs**: `https://www.drawlib.com/`
- **Quickstart Guide**: `https://www.drawlib.com/quickstart/`
- **API Catalog**: `https://www.drawlib.com/api/`
- **Examples & GitHub**: `https://examples.drawlib.com/` · `github.com/yuichi110/drawlib`

